BEGINNER LEVEL

Python for Beginners

A hands-on introduction to the core building blocks of Python — variables, strings, control flow, loops, and functions.

Implose.gg Course Material · 2026

Table of Contents

Introduction

Python is a high-level, general-purpose programming language known for its readable syntax. It is widely used in web development, data analysis, automation, machine learning, and scripting. Because the language reads almost like English, it has become the most common first language taught in schools and bootcamps.

Why this course?

This material gets you comfortable with the five things you will use in *every* Python program you ever write:

- Storing values in variables and knowing their types
- Working with text (strings)
- Making decisions with `if` statements
- Repeating work with loops
- Wrapping logic into reusable functions

Before you start

You don't need to install anything to follow along — every code block below can be tried in an online editor like **replit.com** or the Python shell. Type the examples yourself instead of copy-pasting; muscle memory matters at this stage.

> **Tip:** Python uses *indentation* (spaces at the start of a line) to group code, instead of braces `{ }`. Be consistent — 4 spaces per level is the standard.

Chapter 1 · Variables & Data Types

A **variable** is a named container that holds a value. You create one by assigning a value with `=`. Python figures out the type automatically — you don't have to declare it.

```
name = "Aaron"      # a string
age = 21            # an integer
height = 1.75       # a float
is_student = True   # a boolean
```

The four basic types

Type	Example	What it represents
`int`	`42`	Whole numbers
`float`	`3.14`	Decimal numbers
`str`	`"hello"`	Text
`bool`	`True` / `False`	Yes / no values

Naming rules

- Can contain letters, digits, and underscores
- Cannot start with a digit
- Cannot contain spaces or hyphens (`my-name` is invalid)
- Are case-sensitive: `age` and `Age` are different

Checking a type

```
x = 3.14
print(type(x))        # <class 'float'>
print(type("hello"))  # <class 'str'>
```

The special value `None`

`None` is Python's way of representing "no value" or "nothing." It's commonly used as a placeholder when a value hasn't been set yet.

```
winner = None
print(winner)   # None
```

> **Watch out:** Python's standard library does *not* have a built-in `array` type for everyday use. Beginners usually want a `list` instead (covered later).

Try it

Create three variables — your name, your age, and whether you've eaten today — and print each one along with its type.

Chapter recap

Variables hold values. The four basic types are `int`, `float`, `str`, `bool`. Use `type()` to check a type, and `None` to mean "no value."

Chapter 2 · Strings

A **string** is a sequence of characters wrapped in single or double quotes. Strings are one of the most common data types you'll work with — anything text-based is a string.

```
greeting = "Hello, world!"
name = 'Aaron'
empty = ""
```

Indexing — pick one character

Strings are zero-indexed: the first character is at position `0`. Negative indexes count from the end.

```
s = "python"
print(s[0])    # 'p'
print(s[1])    # 'y'
print(s[-1])   # 'n'  (last char)
```

Slicing — pick a range

The syntax is `s[start:end]` — `start` is included, `end` is not.

```
s = "programming"
print(s[0:4])    # 'prog'
print(s[4:])     # 'ramming'   (until the end)
print(s[-3:])    # 'ing'        (last 3 chars)
```

Common string methods

Method	Example	Result
`.upper()`	`"hi".upper()`	`'HI'`
`.lower()`	`"HI".lower()`	`'hi'`
`.strip()`	`" hi ".strip()`	`'hi'`
`.replace(a, b)`	`"cat".replace("c", "b")`	`'bat'`
`.split(sep)`	`"a,b,c".split(",")`	`['a','b','c']`

Concatenation & length

```
first = "abc"
second = "123"
combined = first + second   # 'abc123'
```

```
print(len(combined))        # 6
```

f-strings — insert values into text

```
name = "Aaron"
age = 21
print(f"My name is {name} and I am {age} years old.")
# My name is Aaron and I am 21 years old.
```

> **Watch out:** Strings are *immutable*. You can't change a character in place — `s[0] = "P"` will raise an error. Build a new string instead.

Try it

Given `email = " Aaron@Example.COM "`, produce the cleaned, lowercase version `"aaron@example.com"` using two string methods chained together.

Chapter recap

Strings are sequences of characters. Use `s[i]` to index, `s[a:b]` to slice, `len(s)` for length, and methods like `.upper()` / `.strip()` to transform them. f-strings make it easy to insert values into text.

Chapter 3 · Control Flow (if / elif / else)

Programs make decisions using **conditional statements**. The keyword `if` runs a block of code only when a condition is true.

```
age = 18
if age >= 18:
    print("You can vote.")
```

elif and else

Use `elif` ("else if") to chain conditions, and `else` for the fallback case.

```
score = 75

if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "F"

print(grade)   # B
```

Comparison operators

Operator	Meaning
`==`	equal to
`!=`	not equal to
`<` `>`	less than / greater than
`<=` `>=`	less than or equal / greater than or equal

Logical operators

Combine conditions with `and`, `or`, and `not`.

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")
```

```
if not has_id:
    print("Please show ID")
```

The ternary expression

For simple two-way decisions you can write it on one line:

```
status = "yes" if 5 > 3 else "no"
print(status)   # yes
```

> **Watch out:** Python uses `==` for comparison and `=` for assignment. `if x = 5` is a syntax error — use `if x == 5`.

> **Try it**
>
> Write a snippet that asks the user for a number and prints "positive", "negative", or "zero" depending on its value.

> **Chapter recap**
>
> `if`, `elif`, and `else` let your program take different paths. Use comparison operators (`==`, `!=`, `<`, `>`, etc.) and combine them with `and` / `or` / `not`.

Chapter 4 · Loops (for / while)

Loops let you repeat work without copy-pasting code. Python has two main loop types: `for` for iterating over a sequence, and `while` for repeating until a condition becomes false.

The for loop

A `for` loop walks through each item in a sequence.

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)
# apple
# banana
# cherry
```

range() — generate numbers

Most counted loops use `range()` to produce a sequence of integers.

```
for i in range(5):       # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):    # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8  (step = 2)
    print(i)
```

> **Tip:** `range(n)` goes from `0` up to *but not including* `n`. So `range(5)` produces 5 numbers (0–4), not 6.

The while loop

A `while` loop runs as long as its condition stays true.

```
count = 0
while count < 3:
    print("hello")
    count += 1
# hello
# hello
# hello
```

break and continue

`break` exits the loop immediately. `continue` skips to the next iteration.

```
for i in range(10):
    if i == 5:
        break          # stop the loop entirely
    print(i)
# 0 1 2 3 4

for i in range(5):
    if i % 2 == 0:
        continue       # skip even numbers
    print(i)
# 1 3
```

Watch out: A `while` loop whose condition never becomes false will run forever. Always make sure the variable in the condition is being updated inside the loop.

Try it

Use a `for` loop to print only the odd numbers from 1 to 20.

Chapter recap

Use `for` when you know what to iterate over, and `while` when you don't know in advance how many iterations you'll need. `break` exits early; `continue` skips an iteration.

Chapter 5 · Functions

A **function** is a reusable block of code with a name. You define one with `def`, then call it whenever you need it.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Aaron")    # Hello, Aaron!
greet("Mei")      # Hello, Mei!
```

Parameters & arguments

The names in the function definition are called *parameters*. The values you pass when calling are *arguments*.

```
def add(a, b):          # a, b are parameters
    return a + b

result = add(3, 5)      # 3 and 5 are arguments
print(result)           # 8
```

The return statement

`return` sends a value back to whoever called the function. If you omit it, the function returns `None`.

```
def square(n):
    return n * n

def no_return():
    pass

print(square(4))      # 16
print(no_return())    # None
```

Default values

You can give a parameter a default, making it optional at call time.

```
def greet(name, greeting="Hi"):
    print(f"{greeting}, {name}!")

greet("Aaron")              # Hi, Aaron!
greet("Aaron", "Welcome")   # Welcome, Aaron!
```

Why use functions?

- **Reuse** — write logic once, call it from many places
- **Readability** — a good function name explains *what* the code does
- **Isolation** — variables inside a function don't leak to the rest of the program

> **Watch out:** Forgetting `return` is a common bug. If your function "doesn't work," check whether you actually returned the result — printing it is not the same as returning it.

Try it

Write a function `is_even(n)` that returns `True` if `n` is even and `False` otherwise. Hint: use the `%` operator.

Chapter recap

Define functions with `def`, pass values via arguments, and send values back with `return`. Functions are the main tool you have to keep code clean and reusable.

Where to Go Next

You now know enough Python to write small programs that take input, make decisions, repeat work, and organize logic into functions. To keep growing, the natural next topics are:

- **Lists, tuples, and dictionaries** — storing collections of values
- **File I/O** — reading and writing files
- **Error handling** — using `try` / `except` to recover from problems
- **Modules** — using libraries written by other people with `import`
- **Object-oriented programming** — classes and objects

The best way to improve is to build something small and finish it. Pick a tiny project — a number-guessing game, a to-do list, a quiz scorer — and use what you've learned in this course to make it work end to end. Good luck!

> **Final word**
> Programming is a skill, not a body of knowledge. You don't learn it by reading — you learn it by typing, breaking things, and fixing them. Open an editor, write some code, and don't be afraid of the red error messages.